

4.6.2 Observables with RxJS

Angular asynchronous data streams and reactive programming

Core idea: An Observable represents a stream of values or events over time. In Angular, RxJS Observables are used for asynchronous work such as HTTP requests, user events, timers, and real-time data.

4.6.2.1 What is an Observable?

Observables in Angular are powered by the RxJS library. An Observable represents a stream of values or events that can be observed over time. It can emit zero, one, or many values and can also signal an error or completion.

Why Observables are useful in Angular

- Handle asynchronous operations without blocking the application.
- Represent streams of values that arrive over time.
- Receive multiple values from one source.
- Compose asynchronous operations with RxJS operators.
- Work naturally with Angular features such as HttpClient and the async pipe.

Examples of data streams

- HTTP responses and API requests.
- User input and DOM events.
- Timer and interval-based values.
- Real-time updates.
- Application state or data changes.

Rx in RxJS

Rx means Reactive Extensions. RxJS (Reactive Extensions for JavaScript) is a JavaScript library for working with asynchronous data streams using Observables and operators.

4.6.2.2 Observable, Observer and Subscription

An Observable is the data source. An Observer is the consumer that reacts to notifications. A Subscription represents the execution connection created when subscribing and can be used to stop that execution.

Term	Meaning
Observable	Produces a stream of values, errors, or completion notifications.
Observer	Receives notifications through next, error, and complete handlers.
Subscription	Represents the active subscription to an Observable and provides unsubscribe().
Subscriber	The observer-side object through which an Observable sends notifications.

Three main notifications

Notification	Purpose
next(value)	Delivers a value to the subscriber. It can happen many times.
error(error)	Reports a failure and terminates the Observable execution.
complete()	Signals successful completion and terminates the Observable execution.

Basic Observable + Observer

```
import { Observable } from 'rxjs';

const numbers$ = new Observable<number>(subscriber => {
  subscriber.next(10);
  subscriber.next(20);
  subscriber.next(30);
  subscriber.complete();
});

numbers$.subscribe({
  next: value => console.log(value),
  error: error => console.error(error),
  complete: () => console.log('Completed')
});
```

Output:

```
10
20
```

30

Completed

Note: Once an Observable sends `error()` or `complete()`, it does not send further `next()` notifications.

4.6.2.3 subscribe() and unsubscribe()

Calling `subscribe()` creates a subscription to the Observable. The subscription is where the emitted values are received.

Subscribe

```
const subscription = numbers$.subscribe({
  next: value => console.log(value),
  error: error => console.error(error),
  complete: () => console.log('Completed')
});
```

For Observables that continue for a long time, the Subscription may need to be cleaned up when the consumer is destroyed.

Unsubscribe

```
const subscription = dataStream.subscribe(value => {
  console.log(value);
});

// Stop receiving values
subscription.unsubscribe();
```

When is cleanup important?

- Long-lived streams such as `interval()`, `fromEvent()`, and custom streams.
- Subscriptions created manually inside components or services.
- Streams that would otherwise keep running after a component is destroyed.

Important: HttpClient Observables usually complete after the response arrives. Long-lived Observables need more attention to subscription lifetime.

4.6.2.4 Lazy / Cold Observable Concept

An Observable commonly represents the work that should happen when a consumer subscribes. For HTTP, Angular documents HttpClient Observables as cold: no request is sent until the Observable is subscribed.

HttpClient Observable

```
const request$ = this.http.get<Product[]>('/api/products');

// The HTTP request is sent when this is subscribed:
request$.subscribe(products => {
  console.log(products);
});
```

Multiple subscriptions

Subscribing more than once can mean more than one execution, depending on the source. With Angular HttpClient, each subscription generally sends an independent backend request.

Observable vs Promise

Feature	Observable	Promise
Values	Can emit multiple values over time.	Resolves once.
RxJS operators	Large operator ecosystem for stream processing.	No equivalent operator pipeline built in.
Cancellation	Subscription can be cancelled/unsubscribed.	Native Promise API has no unsubscribe().
Angular usage	Widely used by HttpClient and reactive APIs.	Can still be used for one-time async results.

4.6.2.5 Creating Observables

RxJS provides creation functions for common kinds of streams. The course example uses `interval()` to simulate real-time data.

`of()`

`of()`

```
import { of } from 'rxjs';

const numbers$ = of(10, 20, 30);

numbers$.subscribe(value => {
  console.log(value);
});
```

Emits the supplied values and then completes.

`from()`

`from()`

```
import { from } from 'rxjs';

const numbers$ = from([10, 20, 30]);

numbers$.subscribe(value => {
  console.log(value);
});
```

Converts an array or other supported iterable-like input into an Observable sequence.

`interval()`

`interval()`

```
import { interval } from 'rxjs';

const counter$ = interval(1000);

counter$.subscribe(value => {
  console.log(value);
});
```

`interval(1000)` emits sequential numbers at 1000-millisecond intervals. The first value is 0, then 1, 2, 3, and so on.

4.6.2.6 RxJS Operators

RxJS operators are functions used to transform, filter, combine, limit, or otherwise control Observable streams. Operators are commonly used with the `pipe()` method.

`pipe()` with operators

```
const result$ = source$.pipe(
  map(value => value * 2),
  filter(value => value > 5)
);
```

Operator	Use	Example idea
<code>map</code>	Transform each emitted value.	1 -> 2, 2 -> 4, 3 -> 6
<code>filter</code>	Keep only values that meet a condition.	Keep values > 5
<code>take</code>	Take only a fixed number of emissions.	First 4 values
<code>tap</code>	Perform a side effect without changing the emitted value.	Log the value
<code>debounceTime</code>	Wait for a period before emitting the latest value.	Search box input
<code>switchMap</code>	Switch to a new inner Observable and stop using the previous one.	Search/API requests
<code>catchError</code>	Handle an error and optionally return another Observable.	Fallback data
<code>combineLatest</code>	Combine the latest values from multiple Observables.	Latest values from two streams

Operator example

```
import { of } from 'rxjs';
import { map, filter } from 'rxjs/operators';

const numbers$ = of(1, 2, 3, 4, 5);

numbers$
  .pipe(
    map(n => n * 2),
    filter(n => n > 5)
  )
  .subscribe(value => console.log(value));
```

Output:

```
6
8
```


4.6.2.7 Real-time Data with interval()

The uploaded course notes demonstrate a DataService with an Observable<number> called `dataStream`. The service creates the stream using `interval(1000)`, and a component subscribes to the stream and updates a `realTimeData` property whenever a new value is emitted.

DataService - course example

```
// src/app/data.service.ts

import { Injectable } from '@angular/core';
import { Observable, interval } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  private dataStream: Observable<number>;

  constructor() {
    this.dataStream = interval(1000);
  }

  getRealTimeData(): Observable<number> {
    return this.dataStream;
  }
}
```

Component - course example

```
// src/app/app.ts

import { Component, OnInit } from '@angular/core';
import { DataService } from './data.service';

@Component({
  selector: 'app-root',
  template: `
    <div>
      <h1>Real-Time Data Streaming with Angular Observables</h1>
      <p>{{ realTimeData }}</p>
    </div>
  `,
})
export class AppComponent implements OnInit {
  realTimeData!: number;

  constructor(private dataService: DataService) {}

  ngOnInit() {
```

```
this.dataService.getRealTimeData().subscribe((data) => {  
  this.realTimeData = data;  
});  
}  
}
```

How the example works

1. DataService creates an interval Observable that emits a number every second.
2. getRealTimeData() returns the Observable to the component.
3. The component receives DataService through dependency injection.
4. ngOnInit() subscribes to the Observable.
5. Every emitted value is assigned to realTimeData.
6. Angular updates the template so the displayed value changes over time.

Note: This course example is intended to simulate real-time data. In a real application, the source could be an API stream, user event, WebSocket-based service, or another asynchronous source.

4.6.2.8 Using the Async Pipe

Angular's AsyncPipe subscribes to an Observable or Promise in the template, displays the latest value, and handles subscription cleanup when the view is destroyed.

AsyncPipe example

```
import { Component } from '@angular/core';
import { AsyncPipe } from '@angular/common';
import { interval } from 'rxjs';

@Component({
  selector: 'app-counter',
  standalone: true,
  imports: [AsyncPipe],
  template: `
    <h2>Counter</h2>
    <p>{{ counter$ | async }}</p>
  `,
})
export class CounterComponent {
  counter$ = interval(1000);
}
```

Manual subscription vs AsyncPipe

Manual subscription	AsyncPipe
Uses subscribe() in component/service code.	Subscribes in the template.
Developer manages the resulting value.	Angular updates the template with emitted values.
Long-lived subscriptions may need explicit cleanup.	Angular manages the pipe subscription lifecycle.
Useful when imperative logic is required.	Useful when the Observable is mainly for display.

Note: Angular's documentation recommends the async pipe or toSignal() when appropriate because they help dispose of subscriptions correctly.

4.6.2.9 Observable and Subject - Basic Difference

A Subject is both an Observable and an Observer. It can receive values through `next()` and broadcast those values to its current subscribers. The core course topic is Observable, but Subjects are commonly encountered in Angular applications.

Observable	Subject
Usually represents a producer of values.	Can act as both producer and multicast subscriber target.
Consumers subscribe to receive values.	Subscribers receive values pushed through <code>next()</code> .
Common for HTTP and derived data streams.	Useful for sharing events or values among multiple consumers.

Basic Subject example

```
import { Subject } from 'rxjs';

const message$ = new Subject<string>();

message$.subscribe(value => console.log('A:', value));
message$.subscribe(value => console.log('B:', value));

message$.next('Hello');
message$.next('Angular');
```

Both subscribers receive the emitted values.

Common Angular uses for RxJS

- HTTP data access with `HttpClient`.
- Search boxes with `debounceTime()` and `switchMap()`.
- Component communication with shared streams.
- Real-time counters and timers with `interval()`.
- Combining related asynchronous data with `combineLatest()` or `forkJoin()`.

Quick Revision

Topic	Remember
Observable	A stream of values/events over time.
RxJS	Reactive Extensions for JavaScript; used heavily for asynchronous streams in Angular.
next()	Emits a value to subscribers.
error()	Signals an error and ends the stream.
complete()	Signals successful completion.
subscribe()	Subscribes to an Observable and receives notifications.
unsubscribe()	Stops a subscription.
interval()	Creates an Observable that emits sequential numbers periodically.
pipe()	Chains RxJS operators.
map	Transforms values.
filter	Keeps values matching a condition.
take	Limits the number of emitted values.
switchMap	Switches to a new inner Observable.
AsyncPipe	Subscribes in the template and manages the subscription lifecycle.

Important code patterns

Exam revision sheet

```
// Observable
const data$ = new Observable<number>(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.complete();
});

// Subscribe
data$.subscribe({
  next: value => console.log(value),
  error: error => console.error(error),
  complete: () => console.log('Done')
});

// Operator pipeline
data$
  .pipe(
    map(value => value * 10),
    filter(value => value > 10)
  )
```

```
.subscribe(value => console.log(value));

// Interval
const counter$ = interval(1000);

// AsyncPipe
{{ counter$ | async }}
```

Note: The course section on pages 53-55 focuses on Observables with RxJS and a real-time interval() example. The notes here keep that structure and add essential Observable, subscription, operator, and AsyncPipe concepts for study.